CS2614 — Computer System Architecture

Chapter 1: Digital Logic Circuits

Based on M. Morris Mano, Computer System Architecture, 3rd ed.

1 Digital Computers and Binary Information

A digital computer represents information using variables that take a limited number of discrete values. In practice, digital computers use two states, represented by the binary digits 0 and 1. A binary digit is called a *bit*.

Groups of bits may represent numbers, characters, instructions, or control information.

WHITEBOARD

---

digital ⇒ discrete values

binary ⇒ $\{0, 1\}$

binary digit = bit

Information is represented by groups of bits.

For example, when interpreted as a binary number,

$$1001011_2 = 1(2^6) + 1(2^3) + 1(2^1) + 1(2^0) = 75_{10}.$$

The same physical bit pattern may have a different meaning under a different code.

WHITEBOARD

$$1001011_2 = 75_{10}$$

same bits → number, character, or control information

---

2 Hardware, Software, and the Computer System

A computer system consists of hardware and software.

Hardware is the physical equipment. Software consists of programs and data. A sequence of instructions for the computer is called a *program*.

The major hardware portions introduced by Mano are the CPU, memory, and input-output facilities.

WHITEBOARD

See Mano, Fig. 1-1.

Computer System = Hardware + Software

CPU: manipulate data + fetch/execute instructions

Memory: store instructions + data

I/O: communicate with the outside world

Mano distinguishes computer organization, computer design, and computer architecture.

WHITEBOARD

Organization	How hardware operates and connects
Design	What hardware is used and how it is built
Architecture	Structure and behavior seen by the user

3 Logic Gates

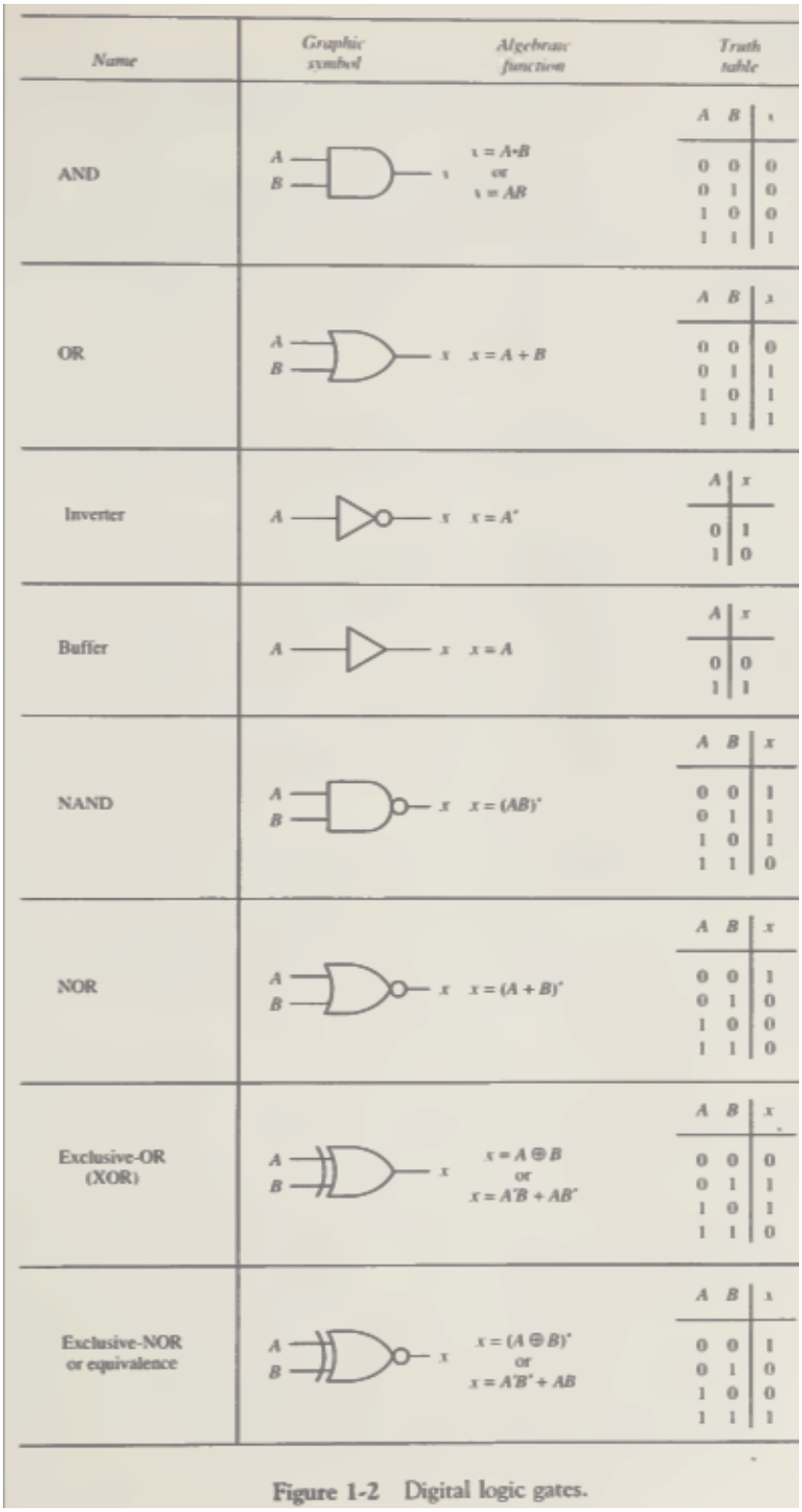
Binary information is represented physically by two recognizable signal states.

Logic gates are hardware blocks that perform logical operations on those binary signals.

A gate can be described by its graphic symbol, Boolean expression, and truth table.

WHITEBOARD

Corresponds to Mano, Fig. 1-2.



Name	Graphic symbol	Algebraic function	Truth table
AND	A, B → x	$x = A \cdot B$ or $x = AB$	A B | x 0 0 | 0 0 1 | 0 1 0 | 0 1 1 | 1
OR	A, B → x	$x = A + B$	A B | x 0 0 | 0 0 1 | 1 1 0 | 1 1 1 | 1
Inverter	A → x	$x = A'$	A | x 0 | 1 1 | 0
Buffer	A → x	$x = A$	A | x 0 | 0 1 | 1
NAND	A, B → x	$x = (AB)'$	A B | x 0 0 | 1 0 1 | 1 1 0 | 1 1 1 | 0
NOR	A, B → x	$x = (A + B)'$	A B | x 0 0 | 1 0 1 | 0 1 0 | 0 1 1 | 0
Exclusive-OR (XOR)	A, B → x	$x = A \oplus B$ or $x = A'B + AB'$	A B | x 0 0 | 0 0 1 | 1 1 0 | 1 1 1 | 0
Exclusive-NOR or equivalence	A, B → x	$x = (A \oplus B)'$ or $x = A'B' + AB$	A B | x 0 0 | 1 0 1 | 0 1 0 | 0 1 1 | 1

Figure 1-2 Digital logic gates.

graphic symbol $\longleftrightarrow$ Boolean expression $\longleftrightarrow$ truth table

The three fundamental operations are AND, OR, and complement.

NAND and NOR are complemented forms of AND and OR. XOR is 1 when its two inputs differ; XNOR is its complement.

For more than two inputs, Mano describes XOR as an *odd function*: the output is 1 when an odd number of inputs are 1.

WHITEBOARD

$\boxed{\text{AND: } x = AB}$ $\boxed{\text{OR: } x = A + B}$ $\boxed{\text{NOT: } x = A'}$

$\boxed{\text{NAND: } x = (AB)'}$ $\boxed{\text{NOR: } x = (A + B)'}$

$\boxed{\text{XOR: } x = A \oplus B}$ $\boxed{\text{XNOR: } x = (A \oplus B)'}$

$$A \oplus B = 1 \iff A \neq B$$

$\boxed{\text{XOR is an odd function.}}$

---

4 Boolean Algebra

Boolean algebra deals with binary variables and the operations AND, OR, and complement.

A Boolean function may be represented algebraically, by a truth table, or by a logic diagram.

For example,

$$F = x + y'z.$$

WHITEBOARD

See Mano, Fig. 1-3.

$\boxed{\text{Boolean variables} \in \{0, 1\}}$

AB (AND), $A + B$ (OR), A' (complement)

$$F = x + y'z$$

$$\boxed{\text{expression} \longleftrightarrow \text{truth table} \longleftrightarrow \text{logic diagram}}$$

The following identities are especially useful when simplifying Boolean expressions.

WHITEBOARD

$$\boxed{\text{Useful Boolean Identities}}$$

$$x + 0 = x, \qquad x \cdot 1 = x, \qquad x + 1 = 1, \qquad x \cdot 0 = 0$$

$$x + x = x, \qquad xx = x, \qquad x + x' = 1, \qquad xx' = 0$$

$$x(y + z) = xy + xz \qquad (x')' = x$$

Simplification matters because a simpler Boolean expression can require fewer logic gates.

Mano's example is

$$F = ABC + ABC' + A'C.$$

Factoring the first two terms gives

$$F = AB(C + C') + A'C = AB + A'C.$$

WHITEBOARD

See Mano, Fig. 1-6.

$$F = ABC + ABC' + A'C$$

$$= AB(C + C') + A'C$$

$$C + C' = 1$$

$$\boxed{F = AB + A'C}$$

$$\boxed{\text{simpler expression} \Rightarrow \text{simpler hardware}}$$

---

5 DeMorgan's Theorems

DeMorgan's theorems are

$$(x + y)' = x'y'$$

and

$$(xy)' = x' + y'.$$

They are important for complementing Boolean expressions and for understanding the alternate forms of NAND and NOR gates.

WHITEBOARD

$\boxed{(x + y)' = x'y'}$ $\qquad$ $\boxed{(xy)' = x' + y'}$

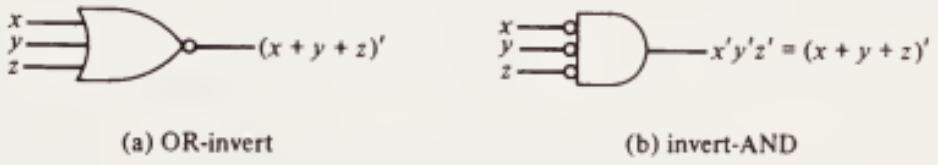
$\boxed{\text{Complement each variable and interchange AND} \leftrightarrow \text{OR}}$

Mano's alternate NOR and NAND symbols are direct applications of DeMorgan's theorem.

WHITEBOARD

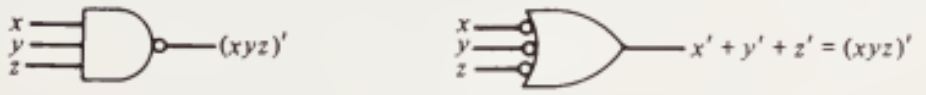
Corresponds to Mano, Fig. 1-4 and 1-5.

Figure 1-4 Two graphic symbols for NOR gate.



(a) OR-invert (b) invert-AND

Figure 1-5 Two graphic symbols for NAND gate.



(a) AND-invert (b) invert-OR

$$\boxed{(x+y)' = x'y'} \qquad \boxed{(xy)' = x' + y'}$$

---

6 Karnaugh Maps

Boolean algebra can simplify expressions, but the next algebraic step is not always obvious.

The Karnaugh map, or K-map, provides a visual procedure for simplification.

Each square represents one minterm. For a function of n binary variables, there are 2^n minterms.

WHITEBOARD

$$\boxed{\text{minterm} = \text{one input combination}}$$

$$n \text{ variables} \Rightarrow 2^n \text{ minterms}$$

$$ABC = 101 \Rightarrow AB'C \Rightarrow m_5$$

$$\boxed{F(A,B,C) = \Sigma(3,4,6,7)}$$

The cells are arranged so that adjacent squares differ in only one variable.

Opposite edges are also adjacent. Groups contain a power-of-two number of cells.

WHITEBOARD

Corresponds to Mano, Fig. 1-7.

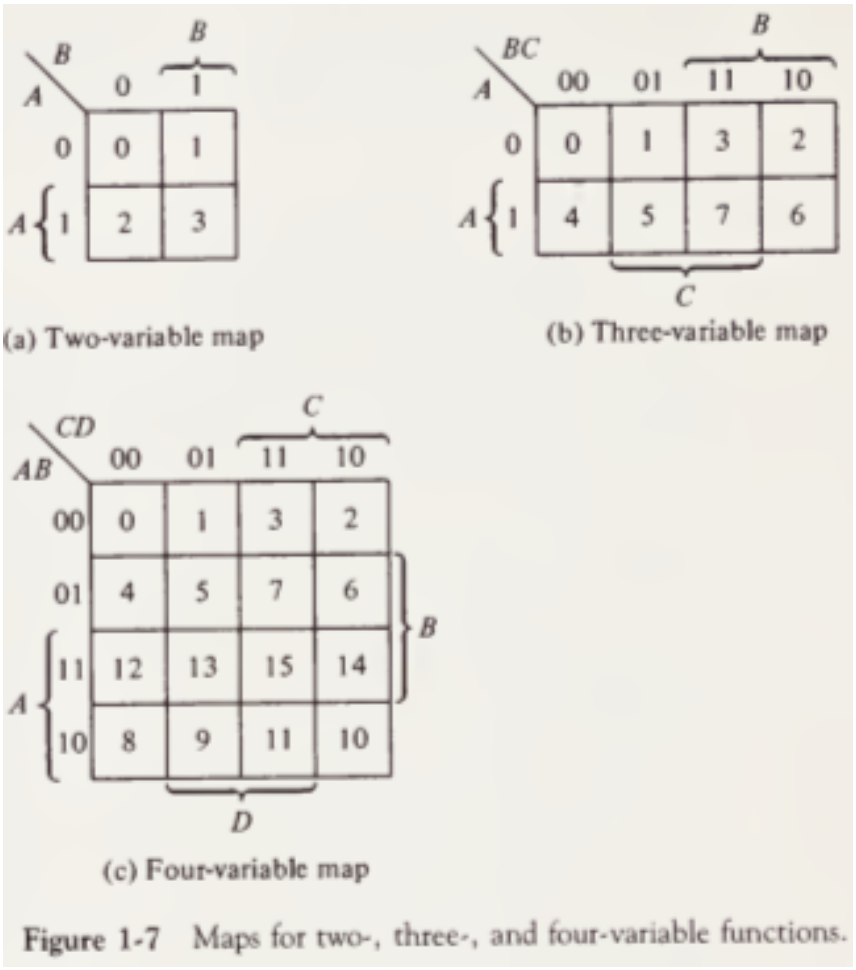


Figure 1-7 Maps for two-, three-, and four-variable functions.

Adjacent squares differ in one variable

Opposite edges are adjacent

Group sizes: $1, 2, 4, 8, \ldots$

6.1 A Basic Three-Variable Example

For Mano's example

$$F(A, B, C) = \Sigma(3, 4, 6, 7),$$

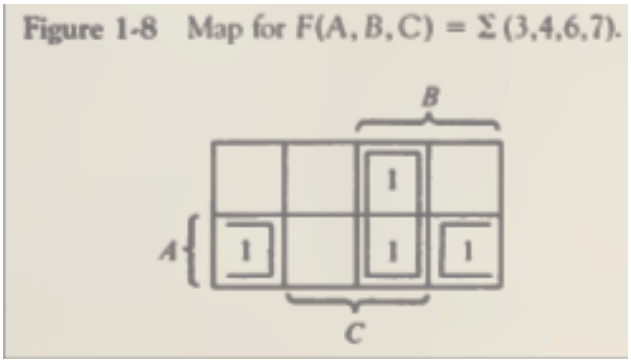
the groups produce the terms BC and AC'.

Therefore,

$$F = BC + AC'.$$

WHITEBOARD

Corresponds to Mano, Fig. 1-8.



$$3, 7 \Rightarrow BC$$

$$4, 6 \Rightarrow AC'$$

$$\boxed{F = BC + AC'}$$

6.2 Standard Forms and Don't-Care Conditions

Grouping 1s gives a sum-of-products form.

Grouping 0s can be used to obtain a product-of-sums form.

A don't-care condition may be taken as either 0 or 1 if that choice simplifies the function.

WHITEBOARD

See Mano, Fig. 1-11 through 1-13.

$$\boxed{\text{Group 1s} \Rightarrow \text{sum of products}}$$

$$\boxed{\text{Group 0s} \Rightarrow \text{product of sums}}$$

$$\boxed{x = \text{don't care: use as 0 or 1 only if helpful}}$$

For Mano's don't-care example,

$$F(A, B, C) = \Sigma(0, 2, 6)$$

and

$$d(A, B, C) = \Sigma(1, 3, 5).$$

Using the useful don't-care entries gives

$$F = A' + BC'.$$

WHITEBOARD

Corresponds to Mano, Fig. 1-14.

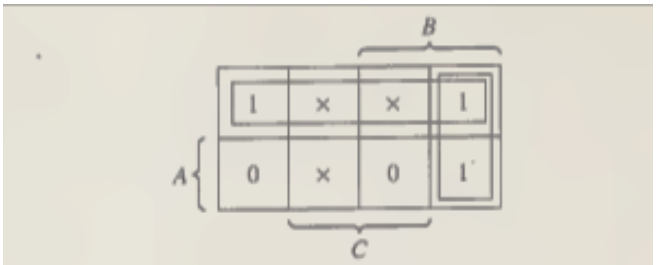


Figure 1-14 Example of map with don't-care conditions.

$$\boxed{F = A' + BC'}$$

---

7 Combinational Circuits

A combinational circuit is an interconnection of logic gates whose outputs at any time are functions only of the present inputs.

It contains no stored state.

WHITEBOARD

See Mano, Fig. 1-15.

$$\boxed{\text{Combinational circuit: Output} = f(\text{present inputs})}$$

No stored state

Mano distinguishes analysis from design.

Analysis begins with a circuit and finds its Boolean functions or truth table.

Design begins with a problem statement and ends with a logic diagram.

WHITEBOARD

Analysis: circuit → Boolean functions → truth table

Design: problem → variables → truth table → simplify → logic diagram

---

8 Half-Adder and Full-Adder

A half-adder adds two binary digits x and y.

It produces a sum S and carry C.

$$S = x \oplus y, \qquad C = xy.$$

WHITEBOARD

See Mano, Fig. 1-16.

Half-Adder

x	y	C	S
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

$S = x \oplus y$ $\qquad$ $C = xy$

A full-adder adds two significant bits and a carry from the previous lower position.

Mano uses inputs x, y, and z, with outputs S and C.

$$S = x \oplus y \oplus z$$

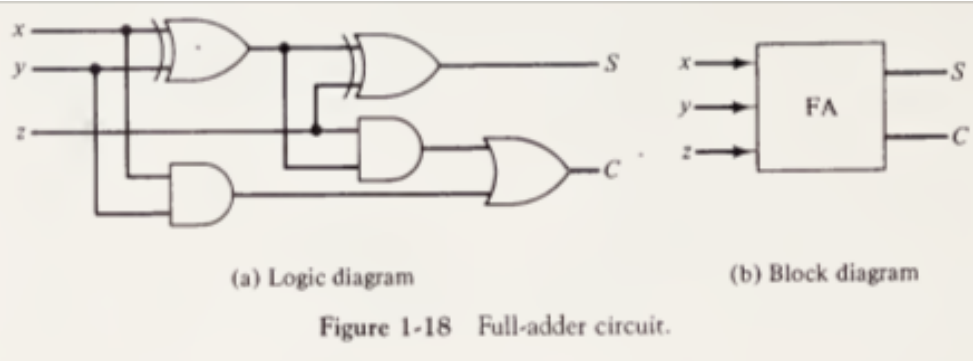
and

$$C = xy + (x \oplus y)z.$$

The circuit can be constructed from two half-adders and an OR gate.

WHITEBOARD

Corresponds to Mano, Fig. 1-18.



(a) Logic diagram (b) Block diagram

Figure 1-18 Full-adder circuit.

$$\boxed{S = x \oplus y \oplus z}$$

$$\boxed{C = xy + (x \oplus y)z}$$

$$\boxed{\text{Full-adder} = \text{two half-adders} + \text{OR}}$$

---

9 Sequential Circuits and Flip-Flops

A sequential circuit contains storage.

Its behavior therefore depends on both the present input and the present state of the storage elements.

WHITEBOARD

$$\boxed{\text{Combinational: } Y = f(X)}$$

$$\boxed{\text{Sequential: } Y = f(X, \text{present state})}$$

$$\boxed{\text{next state} = g(X, \text{present state})}$$

Clocked synchronous sequential circuits use a clock signal to coordinate state changes.

A flip-flop is a binary storage element capable of storing one bit.

WHITEBOARD

$$\boxed{\text{Clock} \Rightarrow \text{synchronizes state changes}}$$

$$\boxed{\text{Flip-flop} = \text{1-bit storage element}}$$

$$Q(t) = \text{present state} \qquad Q(t+1) = \text{next state}$$

9.1 SR Flip-Flop

The SR flip-flop provides set and reset inputs.

The input combination

$$S = R = 1$$

produces an indeterminate next state.

WHITEBOARD

Corresponds to Mano, Fig. 1-19.

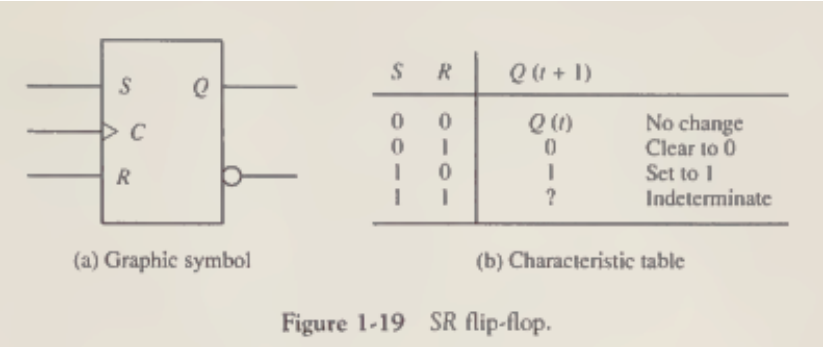


Figure 1-19 SR flip-flop.

$\boxed{S = R = 1 \Rightarrow \text{indeterminate}}$

9.2 D Flip-Flop

For a D flip-flop, the next state is simply the D input sampled at the active clock transition:

$$Q(t+1) = D.$$

WHITEBOARD

Corresponds to Mano, Fig. 1-20.

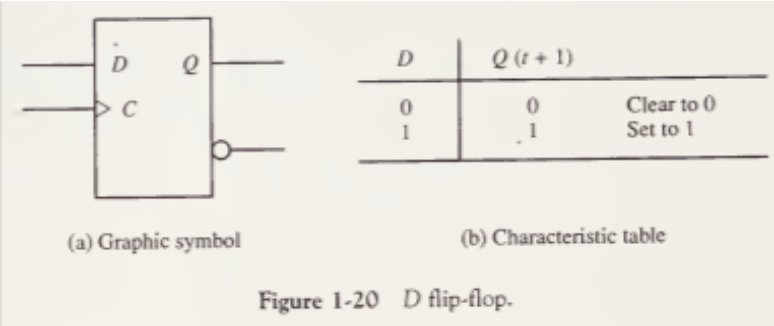


Figure 1-20 D flip-flop.

$\boxed{Q(t+1) = D}$

$\boxed{\text{active clock transition: } Q \leftarrow D}$

9.3 JK and T Flip-Flops

The JK flip-flop replaces the indeterminate SR condition with a toggle.

When

$$J = K = 1,$$

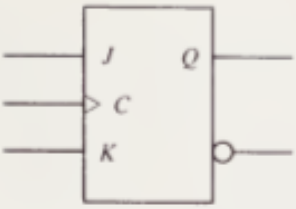
the next state is the complement of the present state.

The T flip-flop has a single toggle input.

When $T = 0$, the state does not change. When $T = 1$, the state complements.

WHITEBOARD

Corresponds to Mano, Fig. 1-21 and 1-22.

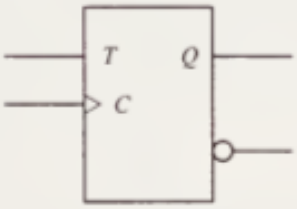


(a) Graphic symbol

J	K	Q (t + 1)	
0	0	Q (t)	No change
0	1	0	Clear to 0
1	0	1	Set to 1
1	1	Q' (t)	Complement

(b) Characteristic table

Figure 1-21 *JK* flip-flop.



(a) Graphic symbol

T	Q (t + 1)	
0	Q (t)	No change
1	Q' (t)	Complement

(b) Characteristic table

Figure 1-22 *T* flip-flop.

$$\boxed{J = K = 1 \Rightarrow \text{toggle}}$$

$$\boxed{Q(t+1) = Q(t) \oplus T}$$

9.4 Edge Triggering

An edge-triggered flip-flop changes state only at a designated transition of the clock.

A positive-edge device responds to

$$0 \to 1,$$

while a negative-edge device responds to

$$1 \to 0.$$

The data input must also satisfy setup-time and hold-time requirements around the active transition.

WHITEBOARD

Corresponds to Mano, Fig. 1-23.

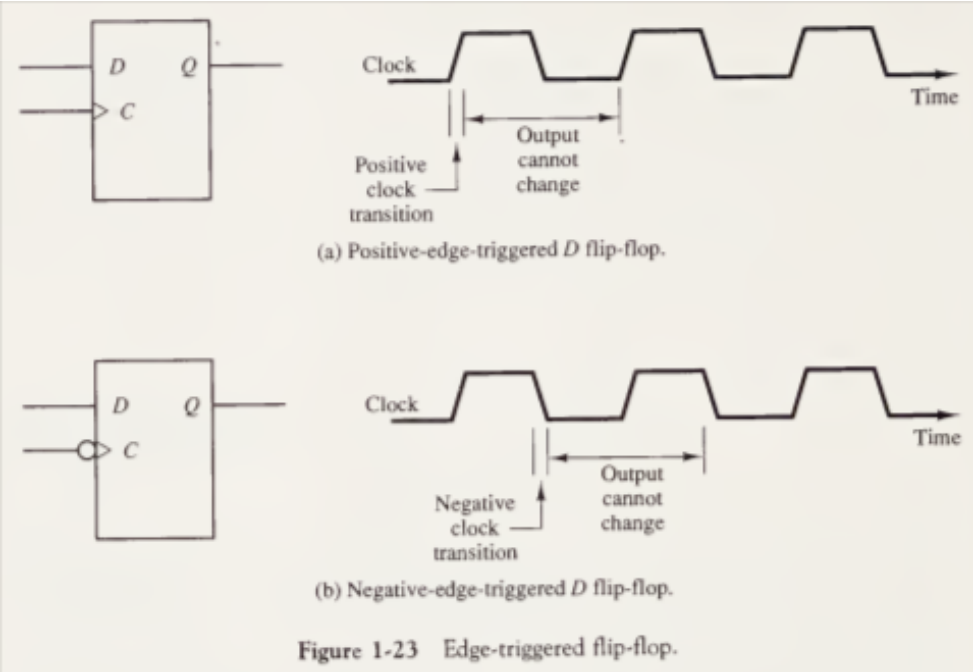


Figure 1-23 Edge-triggered flip-flop.

positive edge: $0 \rightarrow 1$

negative edge: $1 \rightarrow 0$

setup time: input stable before the active edge

hold time: input stable after the active edge

10 Characteristic and Excitation Tables

A characteristic table specifies the next state when the flip-flop inputs and present state are known.

An excitation table works in the reverse direction: it specifies which inputs are needed to obtain a desired state transition.

WHITEBOARD

Characteristic: inputs $+ Q(t) \rightarrow Q(t+1)$

Excitation: $Q(t) + Q(t+1) \rightarrow$ required FF inputs

The JK excitation table is especially useful in Mano's counter design.

WHITEBOARD

JK Excitation Table

$Q(t)$	$Q(t+1)$	J	K
0	0	0	x
0	1	1	x
1	0	x	1
1	1	x	0

$x =$ don't care

---

11 Sequential-Circuit Description and Design

A clocked sequential circuit combines combinational logic and flip-flops.

The combinational part receives external inputs and present-state values. It produces external outputs and the signals that determine the next flip-flop states.

Mano's example with two D flip-flops uses the input equations

$$D_A = Ax + Bx, \qquad D_B = A'x,$$

and the external output

$$y = Ax' + Bx'.$$

WHITEBOARD

See Mano, Fig. 1-24 and 1-25.

next state = f(present state, inputs)

outputs = g(present state, inputs)

$$\boxed{D_A = Ax + Bx} \qquad \boxed{D_B = A'x}$$

$$\boxed{y = Ax' + Bx'}$$

A state table relates present state and input to next state and output.

A state diagram represents the same information graphically.

WHITEBOARD

See Mano, Fig. 1-26.

$$\boxed{\text{present state + input} \rightarrow \text{next state + output}}$$

$$\boxed{\text{state diagram: circle} = \text{state}}$$

$$\boxed{\text{arrow} = \text{transition}} \qquad \boxed{x/y = \text{input/output}}$$

The general design path is:

WHITEBOARD

$$\boxed{\text{specification} \rightarrow \text{state diagram} \rightarrow \text{state table}}$$

$$\boxed{\rightarrow \text{choose FF type} \rightarrow \text{excitation table} \rightarrow \text{K-maps}}$$

$$\boxed{\rightarrow \text{FF input equations} \rightarrow \text{logic circuit}}$$

---

12 Design Example: 2-Bit Binary Counter

Mano's final example is a 2-bit binary counter controlled by an input x.

When $x = 1$, the circuit advances through the binary count sequence.

When $x = 0$, the present state is held.

WHITEBOARD

Corresponds to Mano, Fig. 1-27.

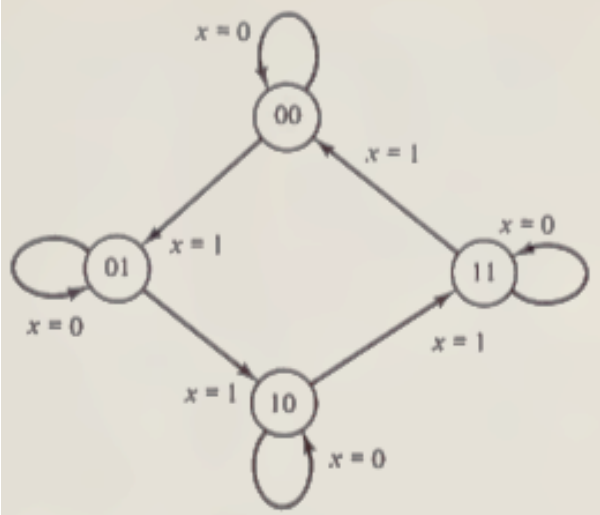


Figure 1-27 State diagram for binary counter.

$$\boxed{x = 1: \quad 00 \to 01 \to 10 \to 11 \to 00}$$

$$\boxed{x = 0: \quad \text{hold present state}}$$

Using the JK excitation table and K-map simplification, Mano obtains the four flip-flop input equations

$$J_A = Bx, \qquad K_A = Bx, \qquad J_B = x, \qquad K_B = x.$$

WHITEBOARD

Corresponds to Mano, Fig. 1-28.

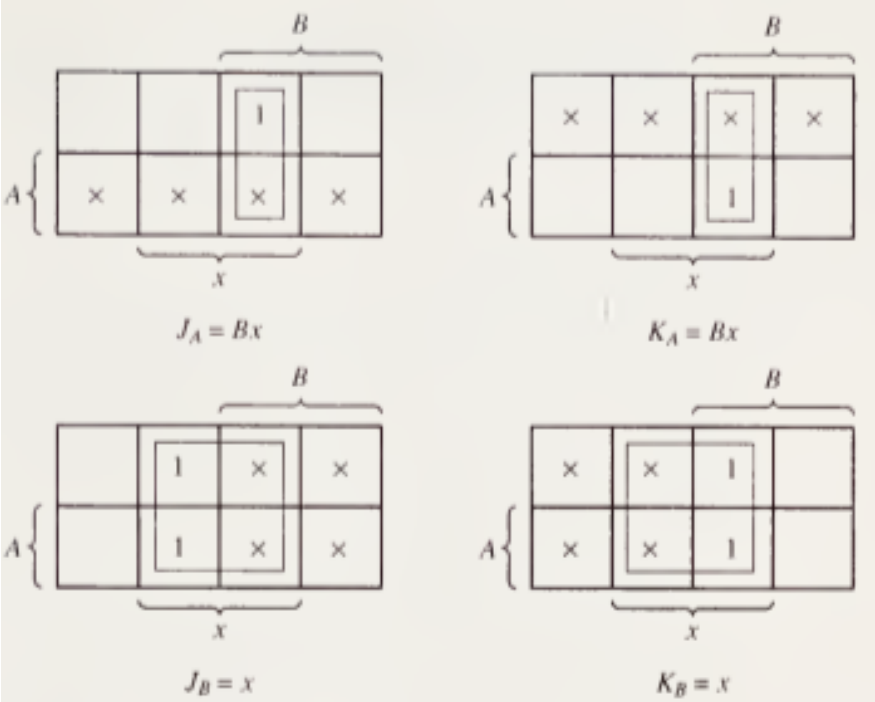


Figure 1-28 Maps for combinational circuit of counter.

$$\boxed{J_A = K_A = Bx}$$

$$\boxed{J_B = K_B = x}$$

The resulting hardware contains two JK flip-flops and one AND gate.

WHITEBOARD

Corresponds to Mano, Fig. 1-29.

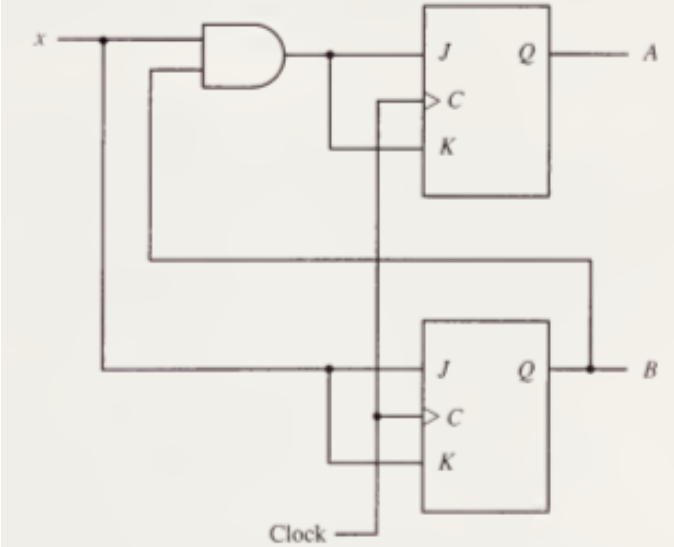


Figure 1-29 Logic diagram of a 2-bit binary counter.

state diagram → state table → excitation table

→ K-maps → FF equations → hardware

---

13 Chapter Summary

Chapter 1 begins with binary information and logic gates, develops Boolean algebra and K-map simplification, uses them to design combinational circuits, and then introduces flip-flops and stored state to build sequential circuits.

WHITEBOARD

---

bits → logic gates → Boolean algebra → K-maps

→ combinational circuits → flip-flops → sequential circuits

Combinational: present input only

Sequential: present input + present state
